Apache HTTP Server

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Floriano Scioscia

Gabriele Colapinto

May 17, 2026

License

Contents

1 Fundamentals of Web Server Configuration **5**
1.1 Core Configuration Concepts: Directives and Contexts 5
1.2 Understanding Apache's Configuration Structure 5

2 The Web Server: A Deep Dive into Apache HTTP Server **9**
2.1 Introduction to Apache 9
2.2 Apache's Architectural Evolution: From Processes to MPMs 9

3 Configuration and Administration **13**
3.1 Installing and Managing Apache HTTP Server on Ubuntu 13
3.1.1 Installing Apache 13
3.1.2 Starting the Apache Service 15
3.1.3 Stopping the Apache Service 15
3.1.4 Restarting the Apache Service 15
3.1.5 Graceful Reload and Graceful Stop 16
3.2 File System Structure 17
3.3 Configuration Hierarchy 17
3.4 Apache Configuration File Syntax 18
3.5 Configuration Contexts and Directives 19
3.5.1 Per-server context 19
3.5.2 Per-Directory Contexts and `.htaccess` Files 22
3.5.3 Example Apache Configuration File 26
3.6 Global configuration directives 27
3.6.1 `ServerRoot` 27
3.6.2 `Timeout` 27
3.6.3 `KeepAlive` 28
3.6.4 `KeepAliveTimeout` 28
3.6.5 `MaxKeepAliveRequests` 29
3.6.6 `ErrorLog` 29
3.6.7 `Include` 30
3.6.8 Interaction Between `KeepAlive`, `KeepAliveTimeout`, and `MaxKeepAliveRequests` 30
3.7 Authorization and Authentication Directives 35
3.7.1 The `AuthType` Directive 35
3.7.2 The `AuthName` Directive 35
3.7.3 `AuthBasicProvider` 36
3.7.4 `AuthUserFile` 36
3.7.5 `Require` 37
3.7.6 Example of Protected Directory Configuration 37
3.8 Managing Access Control and Authentication 38

4 In-Depth Analysis: The Apache HTTP Server **39**
4.1 Apache Modules 39
4.1.1 Built-in Modules 40

4.1.2 Loadable Modules . . . 40
4.1.3 The `LoadModule` Directive . . . 41
4.1.4 Examples of Common Apache Modules . . . 41
4.1.5 Listing Loaded Modules . . . 42
4.1.6 Conditional Configuration with `<IfModule>` . . . 42
4.1.7 Advantages of Apache's Modular Architecture . . . 43
4.2 Serving Dynamic Content . . . 43
4.2.1 Common Gateway Interface (CGI) . . . 44
4.2.2 Server-Side Scripting with Embedded Modules . . . 44

5 Advanced Apache Capabilities 49
5.1 URL Redirection . . . 49
5.2 Virtual Hosting . . . 51
5.2.1 Virtual Host Configuration in Apache . . . 52
5.2.2 Request Processing Pipeline in Apache . . . 53
5.3 Performance Optimization with Caching . . . 55
5.3.1 Caching Policies and Validation Mechanisms . . . 56
5.3.2 Expiration Headers and Freshness Validation . . . 56
5.3.3 Entity Tags (ETags) . . . 58
5.3.4 Example of caching configuration file . . . 59
5.4 Apache as a Proxy Server . . . 60
5.4.1 Forward Proxy . . . 60
5.4.2 Reverse Proxy (Gateway) . . . 61
5.5 Securing Connections with SSL/TLS . . . 63
5.5.1 Core Configuration . . . 63
5.5.2 Generating a self-signed TLS certificate using the Linux `openssl` command 64
5.5.3 Self-Signed Certificates vs External Certification Authorities . . . 65

1 Fundamentals of Web Server Configuration

Web servers like Apache and Nginx are the backbone of the modern internet, but their power and flexibility are unlocked through configuration. These servers rely on text-based configuration files to control their behavior, governing every aspect of its behavior, from how to serve simple static files to managing complex, dynamic applications. For any web developer or systems administrator, understanding the fundamentals of how to read, write, and manage these configurations is a crucial and foundational skill (see figure 1.1).

1.1 Core Configuration Concepts: Directives and Contexts

At the heart of web server configuration lie two simple but powerful concepts: directives and contexts. Understanding this relationship is the first step toward mastering server management.

1. **Directives**: These are the basic commands or settings that instruct the server on how to behave. A directive is a specific instruction, such as setting the location of a log file, defining a security rule, or enabling a specific feature module.
2. **Contexts**: Directives are not applied arbitrarily; they exist within specific scopes or "contexts." A directive might apply globally to the entire server, to a particular website (known as a virtual host), or to a more granular scope like a specific directory or URL path.

1.2 Understanding Apache's Configuration Structure

Apache provides two primary methods for applying configuration directives: a centralized, global file and decentralized, per-directory files. The choice between them has significant implications for performance, security, and manageability.

1. **Main Configuration Files (`httpd.conf`)**: This is the central, global configuration file where server-wide settings and security policies are best established. Directives placed here are loaded once when the server starts, providing optimal performance and a single, authoritative source for server behavior. Centralized management makes it easier to maintain a consistent security posture across the entire server.
2. **.htaccess Files**: These are decentralized configuration files co-located with web content within the document root. They were originally designed to allow individual users on multi-user systems to manage their personal web spaces without needing access to the main server configuration. A directive in an `.htaccess` file can override settings from parent directories or the global configuration (see figure 1.2).

Table 1.1 evaluates the trade-offs between these two approaches.

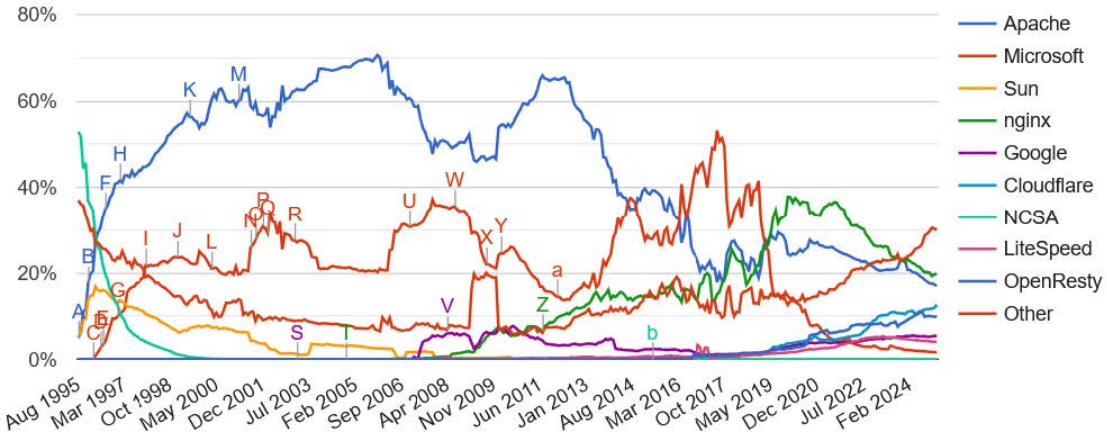

Figure 1.1: Web server market share throughout the years

Feature	httpd.conf (Centralized)	.htaccess (Decentralized)
Performance	**High.** Configuration is loaded once at server startup.	**Lower.** The server must check for `.htaccess` files on *every single request*. This triggers a recursive file system search in the requested directory and all of its parent directories up to the web root, creating a significant and often misunderstood source of I/O overhead.
Security Management	**High.** Security policies are centralized, making them easier to manage, audit, and enforce consistently.	**Lower.** Over-distributing security rules across many `.htaccess` files can make the server's security posture difficult to manage and prone to conflicts or oversights.
Ease of Use	**System Administrator Focused.** Requires root or administrator access to the main server files.	**User Focused.** Allows individual users or developers to make localized configuration changes without server-level access.
When to Use	For global server settings, security policies, performance tuning, and any configuration that should apply server-wide or to an entire virtual host.	Sparingly, for specific, localized overrides when centralized configuration is not feasible, such as content-specific access rules.

Table 1.1: "Comparison of centralized and decentralized configuration"

For optimal performance and manageable security, the architectural best practice is to centralize configuration as much as possible in `httpd.conf` and its included files. The use of `.htaccess` files should be limited to specific corner cases where localized overrides are absolutely necessary.

This document will first explore these concepts in detail through the Apache HTTP Server, one of the most historically significant and widely used web servers in the world.

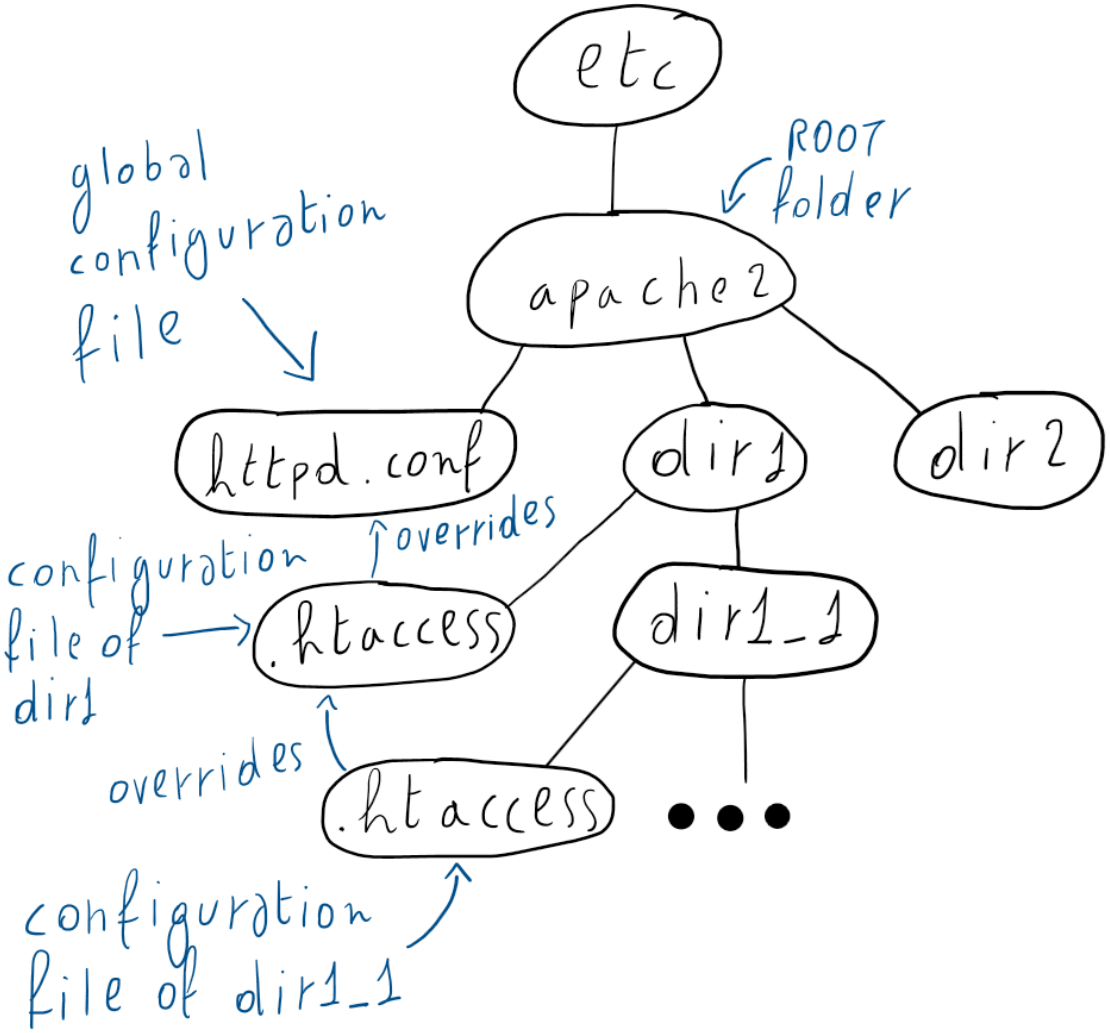

Figure 1.2: Figure showing the global configuration file in the root folder and the local configuration files in the directories `/dir1` and `/dir1/dir1_1`. The figure also highlights the fact that the directives in the configuration file of the `/dir1/dir1_1` folder override the directives in the `/dir1` folder which, in turn, override the directives in the global configuration file.

2 The Web Server: A Deep Dive into Apache HTTP Server

2.1 Introduction to Apache

The Apache HTTP Server is a robust, cross-platform, open-source web server that has played a foundational role in the growth of the World Wide Web. First released in 1995, it remains a significant force in the market with a market share just under 20% for the top one million websites, placing it as one of the top two most-used web servers alongside Nginx. Governed by the Apache Software Foundation, it is known for its stability, flexibility, and extensive feature set.

Apache's primary features include:

- **Concurrency:** The server's architecture places no inherent limits on the number of concurrent client connections it can handle; the practical limit is determined solely by the available hardware and operating system resources.
- **Modular Architecture:** Functionality is implemented through a system of modules that can be enabled or disabled as needed. This allows administrators to customize the server, loading only the features required for a specific application domain, thereby optimizing resource usage. The modularity of the architecture also allows the development of functionalities as add-on modules for specific domains and their addition to the server.
- **Portability:** Apache runs on a wide variety of operating systems, including Unix-like platforms and Windows. This is achieved through the **Apache Portable Runtime (APR)**, an abstraction library that smooths over the differences in operating system APIs for tasks like file and network access and also provides platform specific tuning and optimization.

2.2 Apache's Architectural Evolution: From Processes to MPMs

A significant architectural shift occurred with the release of Apache 2.0, which introduced a more flexible model for handling requests.

- **Legacy Process-Based Model (Pre-2.0):** In earlier versions (e.g., 1.3), Apache operated on a purely process-based model. At startup, it would fork a number of child processes, with each process capable of handling one client connection at a time. This model suffered from two key limitations: the high overhead of creating and context-switching between OS-level processes, and high memory consumption, as isolated processes could not easily share code or data.
- **Modern Multi-Processing Modules (MPMs) Model (2.0+):** Apache 2.0 introduced **Multi-Processing Modules (MPMs)**, which abstract the request-handling architecture. This allows administrators to configure the server in different modes: a pure

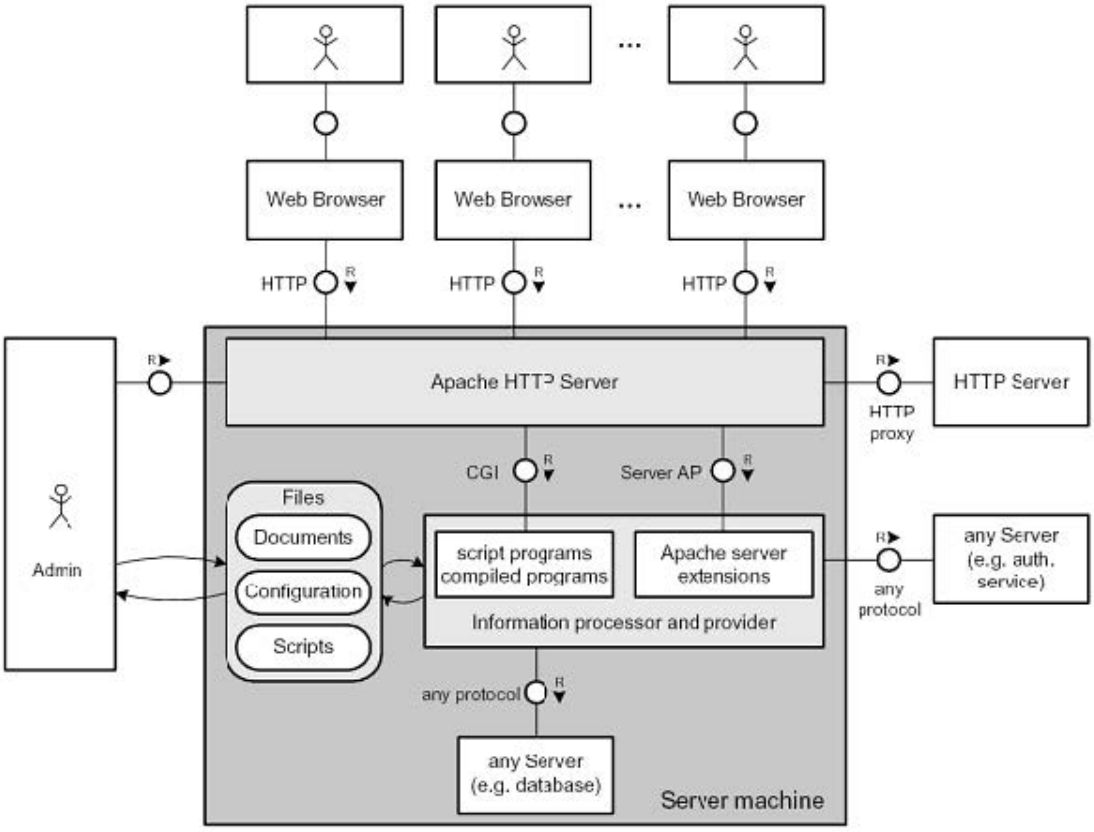

Figure 2.1: Apache HTTP server architecture

process-based model (for legacy compatibility), a pure multi-threaded model, or a hybrid model combining both. Threads inside processes run simultaneously, are more lightweight than processes, they offer faster context switching and the ability to share code and data. On the one hand threaded servers generally scale better than process based servers but, on the other hand, this comes with a trade-off in reliability and security; if one thread misbehaves, it can corrupt data shared by other threads within the same process, potentially impacting multiple independent client connections.

Figure 2.1 shows the architecture of an Apache HTTP server. In the picture we can see the lines starting from a component and terminating in a circle. Those are the components endpoints represented using the UML standard. Those endpoints are paired with a label indicating the protocol used in the communication and a label indicating the type of communication paired with a direction. The label 'R' means request and the arrow points from the requesting entity to the receiver of the request. We don't have to interpret the communications as unidirectional because requesting components expect a response from the consulted entity so, the arrow shows the direction of the first message from the entity initiating the conversation to an entity providing a service. Now, let us move on to the actual figure description.

On the top of the figure we can see the users using their web browser to send HTTP requests

to the Apache Web Server. In the center of the figure we can see the server machine hosting a set of collaborating entities. The entity on top is the Apache HTTP Server which is the entity the users and the administrator interface to.
In the middle there is the content which can be divided in static and dynamic.

- The files section includes both static documents and executable scripts stored on disk:
  - **Documents**: for example hosted files in content distribution platforms such as videos, PDF documents, images, HTML documents, etc...;
  - **Configuration files**: global configuration `httpd.conf` file and local configuration `.htaccess` files;
  - **Scripts**: Source files of the scripts which generate dynamic content when executed.
- Dynamic content is the content generated at runtime changing across the different requests according to data availability, user requests, user preferences, etc... This content is generated by the **information processor and provider** and is divided in:
  - **Script programs / Compiled programs**: Dynamic content generated programmatically at runtime. Script programs are interpreted during execution, while compiled programs are translated into machine code before being executed. This corresponds to the traditional distinction between interpreted and compiled languages;
  - **Apache server extensions**: Dynamically loadable modules extending the functionalities of the Apache HTTP server, such as SSL/TLS support, URL rewriting, proxying, authentication, or scripting language integration.

The Apache HTTP Server interacts with the information processor and provider by using server APIs and Common Gateway Interfaces (CGIs). The Common Gateway Interface (CGI) is a standardized protocol that enables web servers to execute external programs or scripts (CGI scripts) to generate dynamic web content, such as processing form data or querying databases. It acts as middleware, allowing HTTP servers to pass user requests to applications and return the output to the user.

At the bottom of the server machine we can see that it can host additional backend services providing any other type of service and the information processor and provider can interact with the hosted services using a specific protocol for each.

At the left of the server machine we can see that the server administrator can interact with it and they can use their privileged access to change the statically hosted files and at the right we can see that the server machine can make external calls to other services.

We can see that the Apache HTTP server can forward requests to other HTTP servers and act as an HTTP proxy. This functionality is used to make the Apache HTTP server act as a middleman to:

- Mask the external server IP address for security;
- Act as a proxy to provide cached content;
- Act as a gateway to the users;
- Balance the loads across other servers to ensure reliability.

We can also see that the information processor and provider can make external calls to other servers like the ones it can make to other locally hosted services. This basically means that, from

the perspective of the information processor and provider, it makes no difference to receive data from an internally hosted or externally hosted server and the decision to host a server in the machine or delegate the hosting to another company becomes a deployment and infrastructure management decision.

3 Configuration and Administration

Apache's behavior is governed by a system of configuration files containing directives that control its functionality.

Figure 3.1 shows how the three different categories of users interact with the web server.

- **Administrator**: The administrator is responsible for the generation and management of the global configuration files;
- **Authors**: The authors upload documents and static scripts inside their allocated space on the server which is generally a directory. Inside their directory, they can define access policies for their content in a local **.httpaccess** configuration file;
- **Users**: Users can only send HTTP requests to the server to receive its response. They are the consumers of the content and cannot create, edit or delete configuration files.

3.1 Installing and Managing Apache HTTP Server on Ubuntu

Due to its widespread adoption and importance in modern web infrastructures, Apache HTTP Server is included in the official Ubuntu repositories and can be installed and managed through the Ubuntu package management system based on `apt`.

3.1.1 Installing Apache

Apache HTTP Server, together with its documentation and utility programs, can be installed using the following command:

```
sudo apt-get install apache2 apache2-doc apache2-utils
```

The installed packages provide:

- `apache2`: the Apache HTTP Server software;
- `apache2-doc`: the official server documentation;
- `apache2-utils`: a collection of utilities and administrative tools.

After installation, the server configuration files are typically stored in the directory:

```
/etc/apache2
```

while the default web content directory is:

```
/var/www/html
```

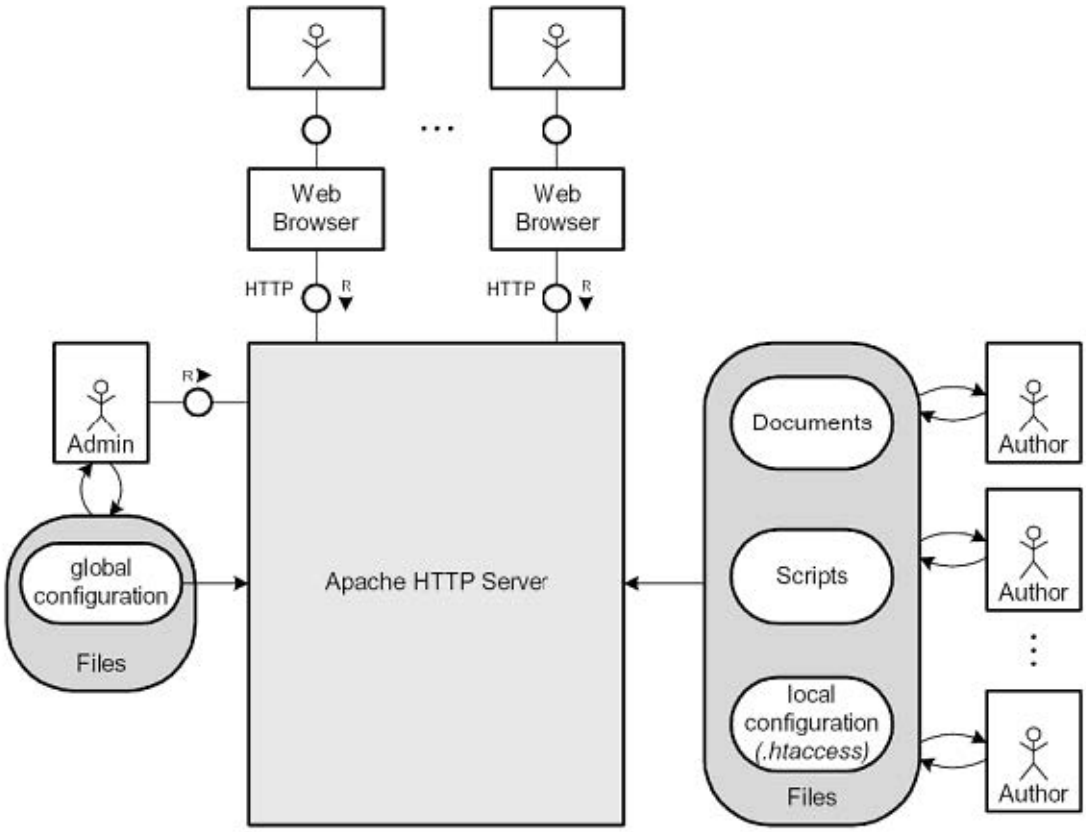

Figure 3.1: Apache HTTP server configuration

3.1.2 Starting the Apache Service

The Apache service can be started using the Ubuntu service manager:

```
sudo service apache2 start
```

Alternatively, the lower-level Apache control utility can be used:

```
sudo apache2ctl -k start
```

If Apache is configured to listen on privileged ports, such as port 80 or port 443, the server must initially be started with root privileges because Unix-like operating systems restrict access to ports below 1024 to privileged users.

For security reasons, however, Apache does not continue serving requests as the root user. After startup, the parent process spawns worker processes running under the unprivileged user:

```
www-data
```

which belongs to the group:

```
www-data
```

This privilege separation mechanism reduces the impact of possible vulnerabilities in the web server or hosted applications.

3.1.3 Stopping the Apache Service

The Apache service can be stopped with:

```
sudo service apache2 stop
```

or equivalently:

```
sudo apache2ctl -k stop
```

This operation sends a termination signal to the Apache parent process, causing it to immediately terminate all child worker processes. As a consequence:

- any requests currently being processed are interrupted;
- no additional client requests are accepted or served.

This type of shutdown is abrupt and may disrupt active client connections.

3.1.4 Restarting the Apache Service

Apache can be restarted using:

```
sudo service apache2 restart
```

or:

```
sudo apache2ctl -k restart
```

A restart operation:

1. terminates the currently running worker processes;
2. reloads the configuration files;
3. reopens log files;
4. spawns a new generation of worker processes.

Unlike a stop operation, the Apache parent process itself does not terminate. Instead, it remains active and supervises the transition between the old and new worker processes. Before performing the restart, Apache automatically executes a syntax check on the configuration files to ensure that no configuration errors are present.

3.1.5 Graceful Reload and Graceful Stop

Apache also supports graceful management operations designed to minimize service interruptions and avoid abruptly terminating active client requests.

3.1.5.1 Graceful Reload

A graceful reload can be performed using:

```
sudo service apache2 reload
```

or:

```
sudo apache2ctl -k graceful
```

During a graceful reload:

- The parent process reloads the configuration files and reopens the log files;
- Existing worker processes are instructed to terminate only after completing their current requests;
- As soon as one of the children worker processes of the previous run terminates, the parent process spawns another with the new configuration.

This mechanism allows Apache to update its configuration without interrupting active connections. As with a normal restart, Apache first performs a configuration syntax validation before applying the new configuration.

3.1.5.2 Graceful Stop

A graceful shutdown can be performed using:

```
sudo -k service graceful-stop
```

or:

```
sudo apache2ctl -k graceful-stop
```

In this mode:

- the parent process stops accepting new incoming connections;
- currently running worker processes continue serving their active requests;
- worker processes terminate only after completing their requests;
- once all workers have terminated, the parent process also exits.

If some worker processes do not terminate within the configured timeout period, Apache forcibly terminates them using the `TERM` signal.

Graceful shutdown procedures are particularly important in production systems because they reduce service interruptions and prevent clients from experiencing incomplete or abruptly interrupted responses.

3.2 File System Structure

On an Ubuntu system, the main files and directories of an Apache HTTP Server installation are organized as follows:

Path	Description
`/usr/sbin/apache2`	Main Apache HTTP Server executable binary. This program starts the web server process and manages incoming HTTP requests.
`/etc/apache2`	Main directory containing the Apache server configuration files, including global settings, module configurations, and virtual host definitions.
`/etc/apache2/conf-available`	Directory containing additional configuration files that can be optionally enabled or disabled. These files are typically used to configure server features, modules, or custom behaviors.
`/etc/apache2/conf-enabled`	Directory containing symbolic links to configuration files stored in `conf-available`. Only linked configurations are loaded by Apache at runtime.
`/etc/apache2/sites-available`	Directory containing configuration files for all available virtual hosts. Each file usually defines the configuration of a specific website or domain hosted by the server.
`/etc/apache2/sites-enabled`	Directory containing symbolic links to virtual host configuration files stored in `sites-available`. Only the linked virtual hosts are enabled and served by Apache.
`/etc/envvars`	File containing environment variables used by Apache during execution, such as user information, process settings, and runtime paths.
`/var/log/apache2`	Default directory containing Apache log files, such as `access.log` and `error.log`, used for monitoring server activity and debugging errors.
`/var/www/html`	Default document root directory containing publicly accessible web resources, such as HTML pages, images, CSS files, JavaScript files, and other static content served by the web server.

3.3 Configuration Hierarchy

There are four ways to configure an Apache Web Server

1. **During building / installation**: Configuring Apache by choosing modules, setting the compiler and flags used for building, selecting the installation path and so on;
2. **At startup**: It is possible to configure the Apache Web Server at startup using command-line options available in the documentation;
3. **Global Configuration:** These are the main server configuration files (e.g., `httpd.conf`) that are processed once at server startup.

4. **Local Configuration:** These are per-directory configuration files named `.htaccess`. These files are processed during a request and allow content authors to customize server behavior for their specific web-space without needing administrative access to the global files.

3.4 Apache Configuration File Syntax

Apache HTTP Server behavior is controlled through configuration files containing a set of instructions called **configuration directives**. These directives define how the server handles requests, manages modules, serves content, and interacts with clients and backend services.

Apache configuration files are processed sequentially, line by line. Empty lines are ignored, while lines beginning with the character **#** are treated as comments and are not interpreted by the server.

A configuration directive generally consists of:

- the **name of the directive**;
- one or more **parameters** associated with the directive.

For example:

```
Listen 80
```

In this case:

- `Listen` is the directive name;
- `80` is the parameter specifying the TCP port used by the server.

Apache supports two main types of directives:

- **Simple directives**: single-line instructions composed of a directive name followed by parameters;
- **Sectioning directives**: block directives used to define a configuration context containing nested directives. These sections are typically delimited by opening and closing tags.

For example:

```
<Directory "/var/www/html">
        Options Indexes FollowSymLinks
        AllowOverride None
</Directory>
```

In this example:

- `<Directory>` is a sectioning directive;
- the enclosed directives apply only to the specified directory.

If a directive requires many parameters, they can be continued onto the next line using the backslash character **** at the end of a line. This improves readability for long configurations.

For example:

```
LogFormat "%h %l %u %t \"%r\" %>s %b" \
combined
```

Apache configuration directives can only be used within specific **contexts**. A context defines where a directive is valid and where its effects apply. Examples of contexts include:

- the global server configuration;
- virtual host definitions;
- directory-specific configurations;
- file-specific configurations.

Each directive is only allowed within a predefined set of contexts. Attempting to use a directive in an invalid context causes Apache to generate a configuration error during startup or reload.

3.5 Configuration Contexts and Directives

To provide fine-grained control, directives are applied within specific **contexts**, which limit their scope. These contexts are divided into per-server contexts and per-directory contexts.

3.5.1 Per-server context

Per-server contexts are defined by following sectioning directives:

- `<VirtualHost>`: Applies directives to a specific virtual server (website). The syntax of this directive is:

  ```
  <VirtualHost addr[:port] [addr[:port]] ...> ... </VirtualHost>
  ```

  The address can be one of the following entities:
  - The IP address of the virtual host (IPv4 or IPv6 enclosed in square braces);
  - A fully qualified domain name (FQDN) for the IP address of the virtual host. Using domain names inside the `<VirtualHost>` directive is not recommended because it requires Apache to perform DNS resolution during startup or configuration reload. This introduces a dependency on the DNS infrastructure, potentially causing delays, configuration errors, or unpredictable behavior if the domain name cannot be resolved correctly. For this reason, specifying explicit IP addresses or using the wildcard character * is generally preferred;
  - The character *, used as a wildcard to match all available IP addresses on the specified port. This is commonly used in name-based virtual hosting configurations.
- `<Directory>`: Applies directives to a specific file system directory and its subdirectories. The syntax of this directive is:

  ```
  <Directory full-directory-path> ... </Directory>
  ```

The main limitation of this directive is that it applies its child directives only to the directory passed as the argument so, to apply the same directives to a set of directories, we would have to copy the same directives for each. To solve this problem, the `<Directory>` has a regular-expression-based counterpart, namely `<DirectoryMatch>`, whose argument is a regular expression instead of a full path.

- `<Files>`: Applies directives to specific files by name. It is comparable to `<Directory>` and has the following syntax:

```
<Files filename> ... </Files>
```

`<Files>` can be nested inside `<Directory>` sections to restrict the portion of the filesystem they apply to and they are processed in the order in which they appear in the configuration file. Also the `<Files>` directive has a regular-expression-based counterpart to address to the same limitations of the `<Directory>` directive which is called `<FilesMatch>`.

- `<Location>`: Applies directives to a specific URL path and its sub-paths. It is comparable to `<Directory>` and has the following syntax:

```
<Location URL-path|URL> ... </Location>
```

`<Location>` sections are processed in the order they appear in the configuration file, after the `<Directory>` sections and `.htaccess` files are read, and after the `<Files>` sections.

`<Location>` sections operate completely outside the filesystem. This has several consequences. Most importantly, `<Location>` directives should not be used to control access to filesystem locations. Since several different URLs may map to the same filesystem location, such access controls may be circumvented.

The enclosed directives will be applied to the request if the path component of the URL meets any of the following criteria:

  - The specified location matches exactly the path component of the URL;
  - The specified location, which ends in a forward slash, is a prefix of the path component of the URL (treated as a context root);
  - The specified location, with the addition of a trailing slash, is a prefix of the path component of the URL (also treated as a context root).

Also the `<Location>` directive has a regular-expression-based counterpart called `<LocationMatch>` that addresses the same limitations of the `<Directory>` and `<Files>` directives.

3.5.1.1 Directive Processing Order and Configuration Merging in the per-server context

When multiple configuration sections apply to the same request, Apache combines their directives according to a well-defined processing and merging order. Understanding this priority system is important because the final server behavior depends on how directives from different contexts interact.

In general, Apache processes configuration sections in the following order:

1. `<Directory>` sections and `.htaccess` files;
2. `<Files>` sections;
3. `<Location>` sections.

This means that:

- filesystem-based access control is evaluated before URL-based access control;
- directives applied to specific files override more general directory configurations;
- URL-based configurations are processed last.

3.5.1.1.1 Directory Context Processing

Among `<Directory>` sections, Apache processes shorter and more general paths first, followed by more specific nested paths.

For example:

```
<Directory "/var/www">
        ...
</Directory>

<Directory "/var/www/private">
        ...
</Directory>
```

In this case, directives defined for `/var/www/private` override conflicting directives inherited from `/var/www`.

If enabled, `.htaccess` files are processed after the corresponding `<Directory>` sections and can further override configuration settings depending on the value of the `AllowOverride` directive.

3.5.1.1.2 Files Context Processing

After directory-based processing, Apache evaluates `<Files>` and `<FilesMatch>` sections. These directives apply to specific filenames independently of the directory hierarchy.

For example:

```
<Files "secret.txt">
        Require all denied
</Files>
```

This configuration applies to every file named `secret.txt` within the directories affected by the current request.

3.5.1.1.3 Location Context Processing

Finally, Apache evaluates `<Location>` and `<LocationMatch>` sections. These directives operate on URL paths rather than filesystem paths.

Since multiple URLs may map to the same filesystem resource, `<Location>` sections should generally not be used for filesystem security restrictions. Instead, they are more appropriate for configuring web application endpoints, proxies, or dynamically generated resources.

3.5.1.1.4 Nested Sections

Apache also supports nested configuration sections. In nested configurations, the inner section inherits the directives of the outer section and may override them when conflicts occur.

For example:

```
<Directory "/var/www">
        Require all granted

        <Files "admin.txt">
                Require all denied
        </Files>
</Directory>
```

In this case:

- all files in `/var/www` are accessible;
- except the file `admin.txt`, whose access is explicitly denied.

3.5.2 Per-Directory Contexts and `.htaccess` Files

Apache HTTP Server supports a special per-directory configuration mechanism through the use of `.htaccess` files. These files allow local configuration changes to be applied to a specific directory and all of its subdirectories without modifying the main server configuration files.

The `.htaccess` files follow the same syntax rules as the main Apache configuration files:

- configuration directives are processed line by line;
- empty lines are ignored;
- lines beginning with the character **#** are treated as comments;
- directives may be either simple directives or sectioning directives.

Directives defined inside a `.htaccess` file apply only to the directory in which the file is placed and to all of its descendant subdirectories.

3.5.2.1 Directory Walk and Configuration Merging

Whenever Apache determines that the requested resource corresponds to a file stored on the filesystem, it starts an internal procedure known as the **directory walk**. During this process, Apache traverses the directory hierarchy associated with the requested resource in order to

determine all configuration directives that apply to the request. The purpose of the directory walk is to collect and merge configuration settings originating from:

- the global Apache configuration files;
- `<Directory>` configuration containers;
- possible `.htaccess` files encountered along the filesystem path.

Apache conceptually starts from the root directory of the filesystem and progressively descends through each directory composing the resource path until it reaches the directory containing the requested file. For example, if the requested resource is:

```
/var/www/html/project/index.php
```

Apache evaluates the configuration associated with:

```
/
/var
/var/www
/var/www/html
/var/www/html/project
```

At each step, Apache checks:

- whether matching `<Directory>` directives exist in the global configuration files;
- whether a `.htaccess` file is present and allowed by the current server configuration.

Whenever Apache encounters a new applicable configuration context, the newly discovered directives are merged with the configuration state accumulated so far. Consequently, the final configuration applied to the requested resource is the result of combining:

- global server directives;
- parent directory configurations;
- subdirectory-specific overrides;
- local `.htaccess` directives.

This mechanism allows configuration inheritance along the filesystem hierarchy. Therefore, directives defined in subdirectories may override or extend directives inherited from ancestor directories, enabling fine-grained customization of server behavior for specific portions of a website.

The directory walk mechanism is one of the key features that makes Apache highly flexible, although extensive use of `.htaccess` files may negatively impact performance because Apache must repeatedly search for and parse these files during request processing.

3.5.2.2 The `AllowOverride` Directive

The server administrator controls whether `.htaccess` files are allowed and which categories of directives may be used inside them through the global `AllowOverride` directive configured in the main Apache configuration files.

The `AllowOverride` directive specifies which groups of directives are permitted within per-directory contexts. The main categories are:

- **AuthConfig**: directives controlling authentication and authorization mechanisms;
- **Limit**: directives controlling access restrictions and request authorization;
- **Options**: directives controlling specific directory features and behaviors. If multiple options could apply to a directory, then the most specific one is used and the others are ignored. The options are not merged by default unless they are preceded by a `+` or `-` symbol.
- **FileInfo**: directives controlling document metadata, MIME types, URL rewriting, and other file-related attributes;
- **Indexes**: directives controlling automatic directory indexing behavior.

For example:

```
<Directory "/var/www/html">
        AllowOverride AuthConfig Indexes
</Directory>
```

In this case, only authentication-related and indexing-related directives may be used inside `.htaccess` files located within the specified directory hierarchy.

If `AllowOverride None` is specified, Apache completely ignores all `.htaccess` files in that directory tree.

3.5.2.2.1 Merging `Options` Directives

The `Options` directive deserves special attention because its behavior differs from most Apache directives. Consider the following configuration:

```
<Directory "/var/www">
        Options Indexes FollowSymLinks
</Directory>

<Directory "/var/www/private">
        Options ExecCGI
</Directory>
```

In this case, the second `Options` directive completely replaces the inherited one for the `/var/www/private` directory. Therefore, only `ExecCGI` remains active inside `/var/www/private`, while `Indexes` and `FollowSymLinks` are discarded.

To explicitly merge or remove inherited options, the `+` and `-` prefixes can be used. For example:

```
<Directory "/var/www">
        Options Indexes FollowSymLinks
</Directory>

<Directory "/var/www/private">
        Options +ExecCGI -Indexes
</Directory>
```

In this configuration:

- `+ExecCGI` adds the `ExecCGI` option to the inherited configuration;
- `-Indexes` removes the `Indexes` option from the inherited configuration;
- `FollowSymLinks` remains enabled because it is inherited and not explicitly removed.

As a result, the final active options for `/var/www/private` become:

- `FollowSymLinks`;
- `ExecCGI`.

3.5.2.3 Directory Indexing

When a client requests a resource corresponding to a directory rather than a specific file, Apache may automatically generate and return a page containing the list of files and subdirectories stored inside that directory. This behavior is controlled by the `Indexes` option. For example:

```
Options Indexes
```

If directory indexing is enabled and no default index file (such as `index.html`) is present, Apache generates an HTML page listing the directory contents.

For security reasons, directory indexing is often disabled in production environments because it may expose sensitive files or internal application structures.

3.5.2.4 Access Control Mechanisms

Apache provides several mechanisms to control access to resources. In Apache 2.2, access control was primarily managed using the combination of the `Order`, `Allow`, and `Deny` directives.

For example:

```
Order deny,allow
Deny from all
Allow from example.org
```

This configuration denies access to all clients except those originating from the domain `example.org`.

Apache 2.4 introduced the more flexible and expressive `Require` directive, which simplifies authorization management and replaces most uses of the older access control directives. Examples include:

- `Require all granted`: grants access to all clients;
- `Require all denied`: denies access to all clients;
- `Require host example.org`: grants access only to clients originating from the specified hostname;
- `Require ip 192.168.1.1`: grants access only to the specified IP address.

For example:

```
<Directory "/var/www/private">
        Require all denied
</Directory>
```

In this case, all access requests targeting the specified directory are denied.

The introduction of the `Require` directive significantly simplified Apache authorization management by providing a more readable and extensible syntax. For example, in `apache2.conf` the following lines prevent `.htaccess` and `.htpasswd` files from being viewed by Web clients.

```
<FilesMatch"^\.ht">
        Require all denied
</FilesMatch>
```

3.5.3 Example Apache Configuration File

The following example shows a simplified Apache configuration file containing comments, simple directives, sectioning directives, multiline directives, and different configuration contexts.

```
# Global server configuration

ServerRoot "/etc/apache2"

# Apache listens on TCP port 80
Listen 80

# Load a module
LoadModule rewrite_module modules/mod_rewrite.so

# Multiline directive example
LogFormat "%h %l %u %t \"%r\" %>s %b" \
combined

# Default document root
DocumentRoot "/var/www/html"

# Directory-specific configuration
<Directory "/var/www/html">
# Enable symbolic links
Options Indexes FollowSymLinks

# Disable .htaccess overrides
AllowOverride None

# Grant access permissions
Require all granted
</Directory>

# Virtual host configuration
```

```
<VirtualHost *:80>
ServerName example.com

DocumentRoot "/var/www/example"

ErrorLog ${APACHE_LOG_DIR}/example_error.log

CustomLog ${APACHE_LOG_DIR}/example_access.log combined
</VirtualHost>
```

3.6 Global configuration directives

Apache HTTP Server behavior is primarily controlled through a set of global configuration directives defined in the main configuration files. These directives affect the overall behavior of the server and define parameters related to resource locations, connection management, logging, request handling, and configuration modularity.

3.6.1 `ServerRoot`

The `ServerRoot` directive specifies the root directory of the Apache server installation and defines the base path used to locate configuration files, log files, runtime resources, and server modules. For example:

```
ServerRoot "/etc/apache2"
```

On Ubuntu systems, the default value is typically:

```
/etc/apache2
```

This directory usually contains:

- the main configuration files;
- module configurations;
- virtual host definitions;
- symbolic links enabling sites and modules;
- auxiliary configuration resources.

Several relative paths used throughout the Apache configuration are resolved with respect to the `ServerRoot` directory.

3.6.2 `Timeout`

The `Timeout` directive defines the maximum amount of time Apache waits for specific network operations before considering the request failed. For example:

```
Timeout 300
```

In this example, Apache waits up to 300 seconds before aborting stalled operations.

The timeout applies to several situations, including:

- receiving client requests;
- transmitting responses;
- communication with backend services or CGI scripts.

A timeout value that is too low may prematurely interrupt slow connections, while a value that is too high may cause server resources to remain occupied for excessive periods of time.

3.6.3 KeepAlive

HTTP is built on top of TCP, and establishing a new TCP connection for every HTTP request introduces additional communication overhead. The `KeepAlive` mechanism allows multiple HTTP requests and responses to reuse the same TCP connection. For example:

```
KeepAlive On
```

When enabled:

- clients can send multiple requests over a single connection;
- the overhead of repeatedly opening and closing TCP connections is reduced;
- page loading performance improves, especially for webpages containing many resources such as images, CSS files, and JavaScript files.

If disabled:

```
KeepAlive Off
```

Apache closes the TCP connection immediately after serving each request.

Persistent connections are particularly important for modern web applications because browsers commonly download multiple resources concurrently from the same server.

3.6.4 KeepAliveTimeout

The `KeepAliveTimeout` directive specifies how long Apache keeps an idle persistent connection open while waiting for additional requests from the same client. For example:

```
KeepAliveTimeout 5
```

In this configuration:

- Apache waits 5 seconds for a subsequent request;
- if no additional request arrives within that period, the connection is closed.

Choosing an appropriate timeout value is important:

- shorter timeouts reduce server resource consumption;
- longer timeouts improve responsiveness for clients issuing multiple requests.

3.6.5 MaxKeepAliveRequests

The `MaxKeepAliveRequests` directive limits the maximum number of requests that may be served over a single persistent connection. For example:

```
MaxKeepAliveRequests 100
```

In this example, Apache closes the connection after serving 100 requests from the same client.

This directive helps:

- prevent excessively long-lived connections;
- balance resource usage among multiple clients;
- reduce the risk of clients monopolizing server resources.

If set to:

```
MaxKeepAliveRequests 0
```

Apache allows an unlimited number of requests per connection.

3.6.6 ErrorLog

The `ErrorLog` directive specifies the file where Apache stores diagnostic information, runtime warnings, and server errors. For example:

```
ErrorLog ${APACHE_LOG_DIR}/error.log
```

The error log typically contains:

- server startup and shutdown messages;
- module loading errors;
- configuration errors;
- runtime failures;
- CGI execution problems;
- authentication and authorization failures.

The `${APACHE_LOG_DIR}` variable is commonly defined in the Apache environment configuration and usually points to:

```
/var/log/apache2
```

Error logs are fundamental for:

- debugging configuration issues;
- monitoring server health;
- troubleshooting application failures;
- performing security analysis.

3.6.7 `Include`

The `Include` directive allows Apache to load additional configuration files from external paths. For example:

```
Include conf/ssl.conf
```

This mechanism improves configuration modularity by allowing the server configuration to be divided into multiple specialized files.

The included files may contain:

- SSL/TLS configurations;
- virtual host definitions;
- module-specific settings;
- authentication configurations;
- proxy configurations.

Apache processes included files as if their content were written directly inside the main configuration file.

Modern Apache installations extensively use this modular configuration approach because it:

- improves maintainability;
- simplifies administration;
- enables dynamic activation and deactivation of features;
- reduces configuration complexity.

3.6.8 Interaction Between `KeepAlive`, `KeepAliveTimeout`, and `MaxKeepAliveRequests`

Apache HTTP Server supports **persistent HTTP connections**, also called **Keep-Alive connections**. Persistent connections allow multiple HTTP requests and responses to be exchanged over the same TCP connection instead of opening a new TCP connection for every request.

The behavior of persistent connections is controlled through the interaction of three directives:

- `KeepAlive`;
- `KeepAliveTimeout`;
- `MaxKeepAliveRequests`.

These directives do not provide alternative mechanisms; instead, they cooperate to control:

- whether persistent connections are enabled;
- how long idle connections remain open;
- how many requests may be served through the same connection.

3.6.8.1 KeepAlive

The `KeepAlive` directive enables or disables persistent connections. For example:

```
KeepAlive On
```

When enabled, Apache allows clients to reuse the same TCP connection for multiple HTTP requests. For example:

```
Browser ---- TCP Connection ---- Apache
                    |
                    |-- GET /index.html
                    |-- GET /style.css
                    |-- GET /logo.png
                    |-- GET /script.js
```

All requests are transmitted over the same TCP connection. If instead:

```
KeepAlive Off
```

Apache closes the connection immediately after each response:

```
GET /index.html --> new TCP connection
GET /style.css  --> new TCP connection
GET /logo.png   --> new TCP connection
```

Disabling persistent connections increases communication overhead because each request requires:

- a new TCP handshake;
- additional socket allocation;
- additional operating system resources.

3.6.8.2 KeepAliveTimeout

The `KeepAliveTimeout` directive specifies how long Apache keeps an idle persistent connection open while waiting for additional requests from the same client. For example:

```
KeepAliveTimeout 5
```

This means that Apache waits up to 5 seconds for another request before closing the connection. For example:

```
KeepAlive On
KeepAliveTimeout 5
```

Possible interaction timeline:

```
t = 0s  Client requests /index.html
t = 1s  Apache sends the response

t = 2s  Client requests /style.css
t = 3s  Apache sends the response

t = 8s  No additional requests received, Apache closes the connection
```

Since the connection remained idle for 5 seconds after the last response, Apache terminates it.

The `KeepAliveTimeout` directive is meaningful only when `KeepAlive` is enabled.

3.6.8.3 `MaxKeepAliveRequests`

The `MaxKeepAliveRequests` directive limits the maximum number of requests that may be served through a single persistent connection.

Example:

```
MaxKeepAliveRequests 100
```

This configuration allows at most 100 requests over the same connection before Apache closes it. For example:

```
KeepAlive On
MaxKeepAliveRequests 3
```

Possible interaction:

```
TCP connection opened

Request 1 --> /index.html
Request 2 --> /style.css
Request 3 --> /logo.png

Maximum request limit reached
Apache closes the connection
```

Even if the connection is still active and the timeout has not expired, Apache terminates the connection after the configured request limit is reached. If:

```
MaxKeepAliveRequests 0
```

Apache allows an unlimited number of requests per connection.

3.6.8.4 Combined Interaction of the Three Directives

The three directives cooperate to fully define persistent connection behavior. For example:

```
KeepAlive On
KeepAliveTimeout 5
MaxKeepAliveRequests 100
```

This configuration means:

- persistent connections are enabled;
- Apache waits up to 5 seconds for additional requests after each response;
- a single TCP connection may serve at most 100 requests.

The connection is closed when one of the following conditions occurs:

- the client remains idle longer than the configured timeout;
- the maximum number of requests is reached;
- the client explicitly closes the connection;
- the server terminates or reloads.

3.6.8.5 Example Scenarios

3.6.8.5.1 Scenario 1: Timeout-Based Connection Closure

```
KeepAlive On
KeepAliveTimeout 3
MaxKeepAliveRequests 100
```

Interaction:

```
Request 1 --> /index.html
Request 2 --> /style.css

Client idle for 4 seconds
Apache closes the connection
```

The connection is closed because the idle timeout expired before reaching the maximum request limit.

3.6.8.5.2 Scenario 2: Request-Limit-Based Connection Closure

```
KeepAlive On
KeepAliveTimeout 30
MaxKeepAliveRequests 2
```

Interaction:

```
Request 1 --> /index.html
Request 2 --> /style.css

Maximum request limit reached
Apache closes the connection
```

In this case, the connection is closed because the request limit is reached, even though the timeout has not expired.

3.6.8.5.3 Scenario 3: KeepAlive Disabled

```
KeepAlive Off
KeepAliveTimeout 10
MaxKeepAliveRequests 100
```

Behavior:

```
Request 1 --> connection opened
Response sent --> connection closed

Request 2 --> new connection opened
Response sent --> connection closed
```

In this configuration:

- `KeepAliveTimeout` becomes irrelevant;
- `MaxKeepAliveRequests` becomes irrelevant;
- every request requires a new TCP connection.

3.6.8.6 Performance Considerations

Persistent connections significantly improve performance because they reduce the overhead associated with repeatedly establishing TCP connections. However, long-lived connections also consume server resources such as:

- memory;
- sockets;
- worker processes or threads.

For this reason, configuring:

- excessively high timeout values may waste resources;
- excessively low timeout values may reduce the benefits of persistent connections.

Choosing appropriate values for these directives therefore represents a trade-off between:

- responsiveness and performance;
- scalability and resource utilization.

3.7 Authorization and Authentication Directives

Apache HTTP Server provides a flexible authentication and authorization framework that can be used to restrict access to specific web resources. These mechanisms are commonly configured inside `.htaccess` files to protect directories, webpages, or administrative areas from unauthorized access.

Authentication verifies the identity of a client attempting to access a resource, while authorization determines whether the authenticated client is allowed to access the requested content.

3.7.1 The `AuthType` Directive

The `AuthType` directive specifies the authentication mechanism used to validate client credentials. For example:

```
AuthType Basic
```

The keyword `Basic` enables the HTTP Basic Authentication mechanism. With Basic Authentication:

- the client sends a username and password to the server;
- the credentials are encoded using Base64;
- the encoded credentials are transmitted inside the HTTP request headers.

For example:

```
Authorization: Basic dXNlcjpwYXNzd29yZA==
```

Base64 is only an encoding mechanism and does not provide encryption or confidentiality protection. Consequently:

- credentials can be easily decoded if intercepted;
- Basic Authentication alone should not be considered secure for protecting confidential data;
- Basic Authentication should normally be used together with HTTPS/TLS encryption.

Without HTTPS, an attacker monitoring network traffic could recover usernames and passwords.

3.7.2 The `AuthName` Directive

The `AuthName` directive specifies the authentication realm shown to the user during the login request. Example:

```
AuthName "Restricted webpage"
```

The realm is displayed by the browser inside the authentication popup dialog and helps users understand which protected area they are trying to access.

3.7.3 `AuthBasicProvider`

The `AuthBasicProvider` directive specifies the backend provider used by Apache to validate user credentials. For example:

```
AuthBasicProvider file
```

The keyword `file` indicates that credentials are stored inside a local password file. Apache also supports alternative authentication providers such as:

- **dbm**: credentials are stored inside a DBM database file;
- **ldap**: credentials are validated through an LDAP directory service.

3.7.3.1 DBM Authentication

DBM (Database Manager) refers to a family of lightweight key-value database systems commonly used for fast local storage and retrieval of structured data. When used as an authentication provider:

- usernames act as keys;
- password hashes act as values;
- Apache performs fast credential lookups directly from the database file.

DBM authentication is generally more efficient than plain text password files when managing large numbers of users.

3.7.3.2 LDAP Authentication

LDAP (Lightweight Directory Access Protocol) is a standardized protocol used to access centralized directory services containing information about users, groups, devices, and organizational resources. When LDAP authentication is enabled:

- Apache delegates credential validation to an external LDAP server;
- user accounts can be centrally managed across multiple systems;
- organizations can integrate Apache authentication with enterprise identity management infrastructures.

LDAP authentication is commonly used in corporate environments together with services such as:

- OpenLDAP;
- Microsoft Active Directory.

3.7.4 `AuthUserFile`

The `AuthUserFile` directive specifies the path to the file containing usernames and password hashes. For example:

```
AuthUserFile /etc/httpd/conf/.htpasswd
```

The referenced file typically stores:

- usernames;
- hashed passwords.

A typical `.htpasswd` file may contain entries such as:

```
alice:$apr1$7F3s...$kP9...
bob:$apr1$Ab9x...$Lm2...
```

Passwords are usually stored as cryptographic hashes rather than plain text. Apache provides utilities such as `htpasswd` to create and manage these authentication files.

3.7.5 `Require`

The `Require` directive defines the authorization policy applied after successful authentication. For example:

```
Require valid-user
```

This directive specifies that:

- only successfully authenticated users may access the protected resource;
- any valid user account defined by the configured authentication provider is accepted.

Apache also supports more restrictive authorization rules, such as:

- requiring specific usernames;
- requiring membership in specific groups;
- restricting access to particular hosts or IP addresses.

3.7.6 Example of Protected Directory Configuration

The following example shows a complete authentication and authorization configuration inside a `.htaccess` file:

```
AuthType Basic

AuthName "Restricted webpage"

AuthBasicProvider file

AuthUserFile /etc/apache2/.htpasswd

Require valid-user
```

In this configuration:

- Apache enables HTTP Basic Authentication;
- users accessing the directory are prompted with the realm “Restricted webpage”;
- credentials are validated using a local password file;

- only authenticated users are allowed to access the protected resources.

When a client attempts to access the protected directory:

1. Apache requests authentication credentials;
2. the browser displays a login dialog;
3. the client sends the username and password;
4. Apache validates the credentials;
5. if authentication succeeds, access is granted.

3.8 Managing Access Control and Authentication

Apache provides robust mechanisms for restricting access to specific directories or resources. A common, though now dated, method is Basic Access Authentication.

1. **Basic Access Authentication**: This mechanism challenges the user for a username and password before granting access. It is typically configured in a `.htaccess` file using the following directives:
   - `AuthType Basic`: Specifies the use of the Basic authentication mechanism.
   - `AuthName "Realm Name"`: Sets the text that appears in the authentication prompt shown to the user. This "realm" name identifies the protected area and helps users understand which set of credentials they need to provide, for example, "Corporate Intranet" or "Admin Section."
   - `AuthUserFile /path/to/.htpasswd`: Points to the file containing the list of valid usernames and their corresponding passwords.
   - `Require valid-user`: Enforces the rule that a user must provide valid credentials from the specified `AuthUserFile` to access the resource.
2. **Security Analysis**: `AuthType Basic` is considered **obsolete for protecting sensitive data**. The credentials are sent from the client to the server in plain text after being Base64 encoded. Base64 is an encoding scheme, not an encryption algorithm, meaning the credentials can be easily decoded by anyone who intercepts the traffic.
3. **Securing Configuration Files**: It is a critical security practice to prevent web clients from accessing configuration files like `.htaccess` and password files like `.htpasswd`. Since these files reside within the web root, they could be downloaded if not explicitly protected. The following directive, placed in the main `httpd.conf` file, uses a regular expression to deny all access to any file whose name begins with `.ht`.

4 In-Depth Analysis: The Apache HTTP Server

The Apache HTTP Server is a highly modular, flexible, and historically dominant web server that has powered a significant portion of the web for decades. Its robust and extensible architecture allows administrators to fine-tune its behavior for a vast array of use cases. This chapter will deconstruct Apache's configuration model, from its modules to advanced capabilities like dynamic content generation, performance caching, and security enforcement.

4.1 Apache Modules

Apache HTTP Server provides a modular architecture that allows functionalities to be added or removed through components called **modules**. Modules extend the capabilities of the server beyond its minimal core functionalities and make Apache highly flexible and customizable.

Through modules, Apache can support:

- HTTPS and SSL/TLS encryption;
- URL rewriting;
- proxying and load balancing;
- authentication and authorization mechanisms;
- scripting language integration;
- server-side includes;
- dynamic content generation;
- logging and monitoring functionalities.

The modular architecture allows administrators to enable only the functionalities required by the deployed applications, reducing:

- memory usage;
- attack surface;
- configuration complexity.

Apache modules are divided into two main categories:

- built-in (static) modules;
- loadable (shared) modules.

4.1.1 Built-in Modules

Built-in modules, also called **static modules**, are compiled directly into the Apache server binary during the build process. These modules:

- are permanently integrated into the Apache executable;
- are automatically loaded every time the server starts;
- cannot be disabled without recompiling Apache.

Because they are embedded into the server binary:

- they do not require separate loading directives;
- they are always available during server execution.

Static modules are typically used for:

- core functionalities;
- low-level request processing;
- essential server features.

Examples of functionalities commonly implemented through static modules include:

- core HTTP request handling;
- configuration parsing;
- process management.

4.1.2 Loadable Modules

Loadable modules, also called **shared modules**, are dynamically loaded at runtime only when required. Unlike static modules:

- they are stored as separate shared object files;
- they can be enabled or disabled without recompiling Apache;
- they provide a more flexible configuration mechanism.

On Linux systems, shared modules are typically stored as:

```
/usr/lib/apache2/modules/
```

or:

```
/usr/lib/httpd/modules/
```

depending on the distribution.

Shared modules usually have the extension `.so` which stands for *shared object*.

4.1.3 The `LoadModule` Directive

Shared modules are loaded through the `LoadModule` directive. For example:

```
LoadModule status_module modules/mod_status.so
```

This directive specifies:

- the symbolic module identifier:

  ```
  status_module
  ```

- the path of the shared library implementing the module:

  ```
  modules/mod_status.so
  ```

When Apache starts:

- the shared object file is loaded into memory;
- the module registers its functionalities inside the server.

The `mod_status` module shown in the example provides monitoring information about:

- active connections;
- worker processes;
- server load;
- request statistics.

4.1.4 Examples of Common Apache Modules

Apache provides a large number of optional modules. Some important examples are:

- `mod_ssl`: provides SSL/TLS support for HTTPS connections;
- `mod_rewrite`: provides URL rewriting capabilities;
- `mod_proxy`: enables proxy and reverse proxy functionalities;
- `mod_php`: integrates the PHP interpreter with Apache;
- `mod_status`: exposes server status information;
- `mod_auth_basic`: implements HTTP Basic Authentication;
- `mod_cgi`: enables execution of CGI scripts.

4.1.5 Listing Loaded Modules

Apache provides command-line utilities to inspect the loaded modules.

To display only the modules statically compiled into Apache:

```
apache2ctl -l
```

The option:

```
-l
```

stands for:

```
compiled-in modules list
```

To display all loaded modules, both static and shared:

```
apache2ctl -M
```

This command shows:

- built-in modules;
- dynamically loaded modules;
- their loading type.

Example output:

```
core_module (static)
so_module (static)
rewrite_module (shared)
ssl_module (shared)
status_module (shared)
```

This information is useful for:

- debugging configuration issues;
- verifying module availability;
- checking whether specific functionalities are enabled.

4.1.6 Conditional Configuration with `<IfModule>`

Apache configurations may depend on the availability of specific modules. To avoid configuration errors when optional modules are absent, Apache provides the `<IfModule>` sectioning directive. For example:

```
<IfModule mod_status.c>
        ExtendedStatus On
</IfModule>
```

The enclosed directives are applied only if the specified module is currently loaded. In this example:

- the `ExtendedStatus` directive is activated only if the `mod_status` module is available;
- if the module is missing, Apache ignores the enclosed directives instead of generating an error.

This mechanism improves:

- configuration portability;
- modularity;
- compatibility across different Apache installations.

The argument of the `<IfModule>` directive is usually:

- the module source filename;
- or the module identifier.

For example:

```
<IfModule mod_ssl.c>
```

checks whether the SSL/TLS module is loaded.

4.1.7 Advantages of Apache's Modular Architecture

Apache's modular architecture provides several important advantages:

- flexibility: functionalities can be enabled only when needed;
- extensibility: third-party modules can be integrated easily;
- maintainability: features are isolated into independent components;
- security: unnecessary functionalities can be disabled;
- scalability: the server can be adapted to many different deployment scenarios.

This architecture is one of the primary reasons why Apache became one of the most widely used web servers in both academic and industrial environments.

4.2 Serving Dynamic Content

While serving static files is a core function, the web's power comes from dynamic content. Apache has evolved to support dynamic generation through two primary mechanisms:

1. Common Gateway Interface (CGI)
2. Server-Side Scripting with Embedded Modules

4.2.1 Common Gateway Interface (CGI)

CGI was the original standard for a web server to interact with external programs also referred to as CGI programs or CGI scripts. Using the `ScriptAlias` directive, a URL path (e.g., `/cgi-bin/`) is mapped to a directory on the server containing executable scripts. For example:

```
ScriptAlias "/cgi-bin/" "/usr/local/apache2/cgi-bin/"
```

This instruction binds the folder referenced using the URL `www.example.org/cgi-bin/` to the internal server path `/usr/local/apache2/cgi-bin`. Apache will assume that all the files contained in this folder are CGI scripts and will try to execute them when they are requested.

When a request for that URL arrives, the server executes the corresponding script and sends its output back to the client. This model, however, has a significant performance overhead, as the server must spawn a new process for the external program for every single request.

4.2.2 Server-Side Scripting with Embedded Modules

To address the performance limitations of CGI, a more efficient model emerged: embedding a language interpreter directly into the Apache server process via a module. PHP is a classic example of this approach. Using a module like `mod_php`, the PHP interpreter becomes part of each Apache worker process spawned by the server. Consequently, every Apache worker is able to directly handle and execute PHP scripts without relying on external processes.

This architecture provides several important advantages:

- **Superior Performance**: By eliminating the need to create a separate process for each request, this model drastically reduces memory and CPU overhead. It is therefore particularly suitable for "PHP-heavy" websites, such as those based on WordPress, Drupal, or Joomla, where many requests involve PHP code execution.
- **Simplified Development**: Server-side scripting languages like PHP allow developers to embed dynamic code snippets directly within standard HTML files. The server processes the file, executes the embedded code, and sends the resulting dynamically generated HTML page to the client, significantly simplifying the development workflow.

Despite these benefits, embedding the interpreter inside Apache also introduces some drawbacks. Since the PHP interpreter is loaded into every Apache worker process, the web server requires more system resources, especially memory.

4.2.2.1 Integrating the PHP interpreter in the Apache Web Server

Figure 4.1 shows the Apache Web Server handling an example of POST request. At the top of the figure we can see the Apache Web Server and the Web Client (browser) with a vertical line underneath each. Those vertical lines are temporal lines starting from the top of the figure and ending at the bottom of the latter.

At the right of the figure there is a legend saying that the script files are represented using the UML document symbol with a green border and the binary executable components are represented using a red rectangle with rounded edges. At the left of the figure there are the document `Hoge.php` and the PHP interpreter.

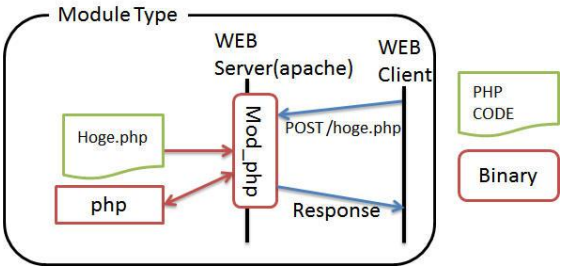

Figure 4.1: Execution of a PHP script using the embedded mod_php module

The fact that the module `mod_php` is overlapped to the timeline of the web server is to highlight the fact that the module binary is integrated in Apache Web Server instead of being an external component.

The client sends a POST request to the server requesting the file `hoge.php`. The web server receives the request and dispatches it to the `mod_php` module which loads the file `Hoge.php` from the disk. At this point, the PHP file is parsed and executed by the PHP runtime which returns execution results, generated output and possible runtime errors to `mod_php`. After completely executing the PHP script, the overall result of the operation is returned to the user in a response message.

The figure is misleading because it treats the PHP interpreter and the `mod_php` module as different components when, in reality, the PHP interpreter is inside the module to make it embedded. The arrows in the figure do not represent a message exchange between different components but, instead, they represent the execution of instructions and the returning of values, status codes, errors, etc... from the PHP interpreter.

4.2.2.2 Executing PHP scripts through the Common Gateway Interface

Historically, PHP scripts could also be executed through the Common Gateway Interface (CGI). In this legacy approach, each request requiring PHP execution caused the web server to spawn a new external process running the interpreter. While this model keeps code execution separated from the web server — providing certain security advantages — it is highly inefficient because of the overhead associated with process creation.

Figure 4.2 shows the same scenario exposed in the previous subsection but, in this case, the PHP script is executed by an external process. In this case the POST request is forwarded to the external process `php-cgi` which parses and executes the PHP file using the PHP runtime. The figure highlights that the execution does not only involve the PHP file but also an argument vector. Let us clarify the execution process.

4.2.2.3 Argument Vector and Parameter Passing in CGI

In the CGI execution model, the web server executes external applications through operating system process creation primitives such as `execve()`. As a consequence, the CGI application receives the traditional Unix process execution parameters:

- an **argument vector** (`argv[]`);

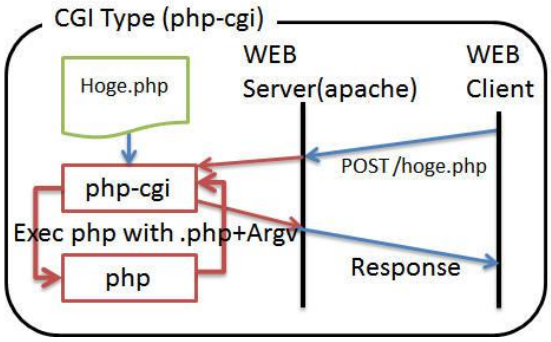

Figure 4.2: Execution of a PHP script using a CGI script

- a set of **environment variables**;
- standard input and output streams (`stdin`/`stdout`).

The figure referring to `php-cgi` mentions the execution of PHP scripts using "`.php + Argv`". This means that Apache invokes the CGI executable while passing the target PHP script as an argument of the process. Conceptually, the invocation may resemble the following:

```
php-cgi /var/www/html/hoge.php
```

Internally, this produces an argument vector similar to:

```
argv[0] = "php-cgi"
argv[1] = "/var/www/html/hoge.php"
```

However, HTTP request parameters are not usually transmitted directly through the argument vector. Instead, the CGI specification maps HTTP information onto Unix process primitives in different ways.

4.2.2.3.1 GET parameters

For HTTP GET requests, the query string contained in the URL is converted into an environment variable named `QUERY_STRING`. For example, the request:

```
GET /hoge.php?name=alice&id=5
```

is transformed by the server into:

```
QUERY_STRING="name=alice&id=5"
```

The PHP interpreter then parses this information to construct the `$_GET` associative array.

4.2.2.3.2 POST parameters

For POST requests, the request body is not passed through the argument vector nor through environment variables. Instead, Apache forwards the HTTP body to the CGI process using the standard input stream (`stdin`). The PHP interpreter reads the input stream and generates the `$_POST` array accordingly.

4.2.2.3.3 HTTP headers

HTTP headers are generally converted into environment variables as well. For instance:

```
User-Agent: Firefox
```

may become:

```
HTTP_USER_AGENT="Firefox"
```

Therefore, although the figure refers to the ”argument vector”, the actual CGI communication model relies on a combination of:

- process arguments;
- environment variables;
- standard input streams.

This design reflects the Unix philosophy underlying CGI, where a web request is translated into the execution of a standard operating system process capable of communicating through conventional Unix I/O mechanisms.

4.2.2.4 Executing PHP scripts using FastCGI

To mitigate these inefficiencies while maintaining process separation, modern systems often adopt **FastCGI**. With FastCGI, PHP scripts are executed by persistent interpreter processes running outside the web server, reducing process creation overhead while still decoupling application execution from the web server itself.

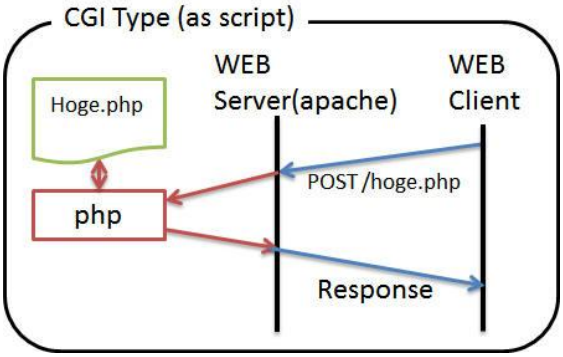

Figure 4.3: Execution of a PHP script using a persistent external interpreter process

Figure 4.3 shows the same POST request as the previous subsections but, in this case, the request is handled by a persistent interpreter process which reads the file `Hoge.php` from disk, executes it and returns the overall response to the Apache Web Server process which, in turn, returns the overall result of the computation to the web client.

If we pay attention to the figure we can notice a small error: the arrow between the file stored on disk and the PHP interpreter is bidirectional. This is wrong because the file stored on disk is a static entity and it doesn't make sense to communicate with it. The arrow connecting the file

on disk and the PHP interpreter should point only to the PHP interpreter.

FastCGI represents an evolution of the traditional CGI model. Instead of spawning a new interpreter process for each request, the web server communicates with persistent external interpreter processes, significantly reducing process creation overhead while maintaining process isolation. This is the foundation of modern high-performance web servers.

5 Advanced Apache Capabilities

A server's core duty is serving content, but its real power lies in managing the entire application delivery ecosystem: scaling horizontally with virtual hosting, optimizing delivery through caching, and routing traffic intelligently with proxies and redirects. Apache provides powerful features to address each of these domains.

5.1 URL Redirection

The `Redirect` directive is used to map an old URL to a new one, instructing the client that a resource has moved. When a client requests the old URL, the server responds with a status code (like `301 Moved Permanently`) and the new location in the `Location` header which is empty if the resource was deleted. The client is then responsible for making a new request to the updated URL. This is essential for maintaining links when site structures change.

To implement a URL redirection, we can use the `Redirect` directive in a configuration file.

```
Redirect [status] [URL-path] URL
```

The `status` argument is the status code of the redirection indicating the type of redirection of the resource. It is also possible to use some predefined strings as the `status` argument which are mapped to the corresponding numerical code. In fact, the most common status codes have a corresponding string for human readability. The possible values for the `status` argument are the following:

- **permanent**: Returns a permanent redirect status (301) indicating that the resource has moved permanently;
- **temp**: Returns a temporary redirect status (302). This is the default;
- **seeother**: Returns a "See Other" status (303) indicating that the resource has been replaced;
- **gone**: Returns a "Gone" status (410) indicating that the resource has been permanently removed. When this status is used the URL argument should be omitted;
- Numbers in the range **300–399**: It is also possible to use codes in the range 300–399 other than 301, 302 and 303 but, in these cases, the URL argument must be present;
- Numbers **outside** the range **300–399**: It is also possible to use numbers outside the range 300–399 and, in these cases, the *URL argument must be omitted*.

The `URL-path` argument is the old absolute path of the document inside the machine which starts with a slash (`/`) indicating the root folder in the server.

The `URL` argument is the new location of the resource. This can either be an absolute path inside the same machine which is expressed in the same format as the previous argument or, it

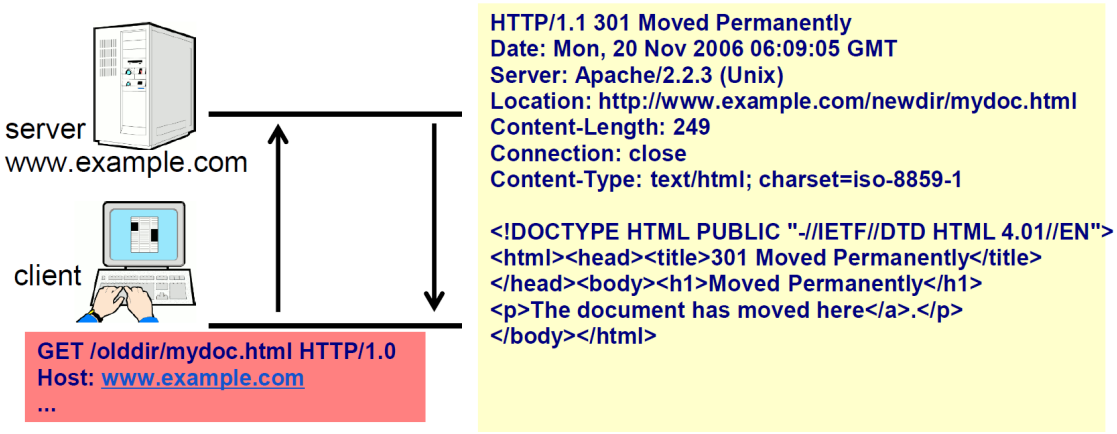

Figure 5.1: Redirection of the request to the document `mydoc.html` from the folder `olddir` to the folder `newdir`. The server response contains a little error because there should be an opening anchor tag (`<a>`) before the word "here".

can also be a location on a different machine or domain expressed using a URL.

Figure 5.1 shows an example of user requesting a permanently moved resource. The user requests the resource referring to its previous location because they don't know about the redirection yet. The old location of the resource is `http://www.example.com/olddir/mydoc.html` and the new location of the resource is `http://www.example.com/newdir/mydoc.html`. When the user asks for the resource in its previous location, the server responds with a message saying that the resource was moved permanently to its new location. The directive that implements this mechanism in the configuration file is the following:

```
Redirect permanent "/olddir/mydoc.html" "/newdir/mydoc.html"
```

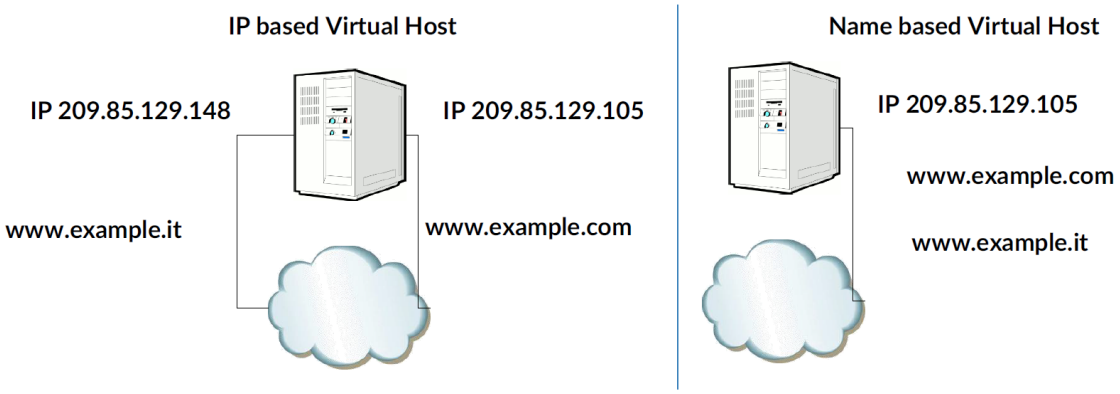

Figure 5.2: Figure showing the difference between the IP based virtual hosting and the name based virtual hosting. In the IP based virtual hosting, we have two different IP addresses for the two websites while, for the name based virtual hosting, the two websites share the same IP address and the server knows which resource to send according to the `Host` header in the client's HTTP request.

5.2 Virtual Hosting

Virtual hosting is the practice of running multiple websites, each with its own domain name, on a single server machine. This is a foundational feature for web hosting and efficient server management. There are two ways of implementing this practice: IP based virtual hosting and name based virtual hosting. Table 5.1 exposes the differences between them and figure 5.2 shows how both of them handle requests to two different websites.

Feature	**IP-Based Virtual Hosting**	**Name-Based Virtual Hosting**
Mechanism	Each website is assigned its own unique IP address. The server determines which site to serve based on the IP address the request was sent to.	Multiple websites share a single IP address. The server uses the `Host` header in the client's HTTP request to determine which domain was requested.
Primary Trade-Off	**Disadvantage:** Requires a dedicated IP address for every website, which is inefficient, costly, administratively burdensome, and contributes to IPv4 address exhaustion.	**Advantage:** Highly efficient and scalable, allowing a nearly unlimited number of sites to run on a single IP address. This is the standard method for virtual hosting.

Table 5.1: Differences between the IP based virtual hosting and the name based virtual hosting

5.2.1 Virtual Host Configuration in Apache

On Ubuntu and Debian systems, Apache adopts a modular configuration structure that simplifies the management of multiple websites hosted on the same server. Instead of placing all virtual host definitions inside a single configuration file, Apache separates site configurations into dedicated directories. The directory:

```
/etc/apache2/sites-available/
```

contains the configuration files for all available websites that can potentially be served by Apache. Each website is typically associated with its own configuration file containing one or more `VirtualHost` blocks.

Among these files, Ubuntu/Debian installations usually provide a default configuration file named:

```
000-default.conf
```

This file defines the default virtual host loaded by Apache. A simplified version of the file is shown below:

```
<VirtualHost *:80>
        ServerAdmin webmaster@localhost
        DocumentRoot /var/www/html

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

We can substitute the asterisk in the snippet with an IP address for the IP based virtual hosting or with a name for the name based virtual hosting.

The prefix `000-` is intentionally chosen so that the file is loaded before other virtual host configurations when Apache processes configuration files alphabetically. As a consequence, this configuration acts as the fallback website whenever no other virtual host matches the incoming request.

The slides suggests that new virtual host configurations should be "based on" `000-default.conf`. This does not necessarily mean that the file itself must be modified. Instead, the common and recommended practice is to use it as a template for creating new configuration files dedicated to individual websites. For example, a new website configuration may be created as:

```
/etc/apache2/sites-available/example.conf
```

containing:

```
<VirtualHost *:80>
        ServerName example.com
        ServerAlias www.example.com

        DocumentRoot /var/www/example
```

```
        ErrorLog ${APACHE_LOG_DIR}/example-error.log
        CustomLog ${APACHE_LOG_DIR}/example-access.log combined
</VirtualHost>
```

In this configuration:

- `ServerName` specifies the primary domain handled by the virtual host;
- `ServerAlias` specifies alternative domain names;
- `DocumentRoot` indicates the directory containing the website files;
- `ErrorLog` and `CustomLog` configure dedicated log files for the website.

Apache distinguishes enabled websites from merely available ones using another directory:

```
/etc/apache2/sites-enabled/
```

A website becomes active only after enabling its configuration through the command:

```
sudo a2ensite example.conf
```

Internally, this command creates a symbolic link from the configuration file stored in `sites-available` to the `sites-enabled` directory. After enabling or modifying a virtual host, Apache must reload its configuration:

```
sudo systemctl reload apache2
```

This modular organization provides several advantages:

- each website remains isolated in its own configuration file;
- websites can be enabled or disabled independently;
- log management becomes simpler;
- server maintenance and debugging are significantly easier.

Therefore, although `000-default.conf` can technically be modified directly, production systems generally avoid using it for custom websites and instead create dedicated configuration files for each virtual host.

5.2.2 Request Processing Pipeline in Apache

After receiving an HTTP request, Apache performs a sequence of internal processing steps in order to determine:

- which virtual host should handle the request;
- which resource is being requested;
- which configuration directives apply to the resource;
- how the final response should be generated.

This procedure is commonly referred to as the **request processing pipeline**.

5.2.2.1 Virtual Host Resolution

The first operation performed by Apache consists in determining the virtual host responsible for handling the incoming request. In name-based virtual hosting configurations, Apache analyzes the value of the HTTP `Host` header sent by the client and compares it with the configured `ServerName` and `ServerAlias` directives.

Once the matching virtual host has been identified, Apache loads the corresponding configuration context associated with that website.

5.2.2.2 Initial Location Walk

Before translating the requested URI into an actual file on disk, Apache performs an initial **location walk**. During this phase, Apache evaluates all `<Location>` configuration blocks whose matching rules apply to the requested URI.

Unlike `<Directory>` containers, which operate on filesystem paths, `<Location>` directives are applied directly to the URI namespace exposed by the web server. This distinction is important because, at this stage, Apache may not yet know whether the requested resource corresponds to a physical file, a dynamically generated resource, or a rewritten URL.

5.2.2.3 URI Translation

After processing the applicable `<Location>` directives, Apache attempts to translate the Request URI into an actual resource. Modules participating in this phase may:

- map the URI to a file on disk;
- redirect the request;
- rewrite the URI;
- forward the request to external handlers.

One of the most common modules involved in this phase is `mod_rewrite`, which allows Apache to dynamically transform incoming URLs according to configurable rewrite rules. For example, a request such as:

```
/products/15
```

may internally be rewritten into:

```
/product.php?id=15
```

without changing the URL visible to the client.

5.2.2.4 File Resolution and File Walk

Once URI translation has completed, Apache determines whether the request corresponds to a physical resource stored on the filesystem. If the target resource is a file, Apache retrieves the configuration directives specifically associated with that file. This phase is commonly referred to as the **file walk**. During this operation, Apache evaluates `<Files>` and related configuration containers that may apply to the selected resource. This mechanism allows administrators to configure rules that target:

- specific filenames;
- file extensions;
- particular classes of resources.

For example, Apache may:

- restrict access to sensitive files;
- apply authentication rules;
- assign MIME types;
- enable or disable script execution for certain extensions.

5.2.2.5 Repeated Location Walk

Some Apache modules may alter the Request URI during processing. When this happens, the rewritten URI may match different `<Location>` rules compared to the original request. Consequently, Apache may perform an additional location walk in order to reevaluate the configuration associated with the new URI. This guarantees that all directives affecting the rewritten request are correctly applied before the final response is generated. The overall request processing pipeline therefore combines:

- virtual host selection;
- URI-based configuration analysis;
- resource translation;
- filesystem-based configuration evaluation;
- dynamic request rewriting.

This layered architecture is one of the main reasons why Apache provides a highly flexible and extensible configuration model.

5.3 Performance Optimization with Caching

Apache includes a sophisticated, HTTP-aware caching system to improve server performance and reduce response times.

1. **The Three-State Cache**: A resource in Apache's cache can be in one of three states:
   - **Fresh**: The content is valid and can be served directly from the cache without further checks.
   - **Stale**: The content's Time-To-Live (TTL) has expired. The server must re-validate it with the origin source before serving.
   - **Non-existent**: The content is not in the cache.
2. **Configuration Directives**: Caching is enabled and configured with a suite of modules and directives:
   - `mod_cache` / `mod_cache_disk`: These are the core modules that enable caching functionality, with `mod_cache_disk` specifically providing disk-based storage.
   - `CacheRoot`: Specifies the local filesystem path where the cached files will be stored.

- `CacheEnable` / `CacheDisable`: These directives control caching for specific URL paths, allowing an administrator to enable caching for static assets while disabling it for dynamic content.
- `CacheMinFileSize` / `CacheMaxFileSize`: These set the lower and upper size limits for files that can be stored in the cache. This prevents wasting cache resources on very small files (where the overhead may outweigh the benefit) or quickly filling the cache with very large files.
- `mod_expires`: This module controls the `Expires` and `Cache-Control: max-age` HTTP headers, which tell clients and proxies how long a resource should be considered fresh (time-based freshness).
- `FileETag`: This directive configures the generation of `ETag` (entity tag) headers, which allow for content-based freshness validation. The client can check if its cached version is still valid by sending the ETag back to the server.

5.3.1 Caching Policies and Validation Mechanisms

Caching does not improve performance in every scenario. When configuring a cache system, administrators must carefully decide which resources should be cached and which should not.

In general, resources are good candidates for caching when:

- they are requested frequently;
- they are relatively small;
- they change infrequently.

Caching very large files may rapidly consume disk space and reduce the effectiveness of the cache. Furthermore, storing resources that are rarely requested may waste cache capacity without providing meaningful performance benefits. For this reason, Apache allows administrators to selectively disable caching for specific resources using directives such as:

```
CacheDisable /update/
```

This mechanism is commonly used for:

- dynamic content;
- frequently updated resources;
- sensitive information;
- files whose caching overhead would outweigh the performance benefits.

5.3.2 Expiration Headers and Freshness Validation

The `mod_expires` module allows Apache to generate HTTP expiration metadata that helps clients and proxy servers determine whether a cached resource is still valid. In particular, the module can automatically generate:

- The `Expires` header. This header contains the time after which the response is considered expired and uses the following syntax:

  ```
  Expires: <day-name>, <day> <month> <year> <hour>:<minute>:<second> GMT
  ```

- The `Cache-Control: max-age` header. Unlike the `Expires` header, which specifies an absolute expiration date and time, the `max-age` directive defines freshness using a relative amount of time expressed in seconds. Its syntax is:

  ```
  Cache-Control: max-age=<seconds>
  ```

  The value associated with `max-age` represents the maximum amount of time during which the resource may be considered fresh starting from the moment the response is generated by the server. For example:

  ```
  Cache-Control: max-age=3600
  ```

  indicates that the resource can be reused from the cache for one hour without contacting the server again. The `Cache-Control` header is generally preferred over the older `Expires` header because:

  - it avoids synchronization problems caused by incorrect client clocks;
  - it is easier to compute since it uses relative time intervals;
  - it provides additional caching directives beyond expiration management.

  The `Cache-Control` header may also contain several other directives controlling cache behavior, such as:

  - `public`: indicates that the response may be cached by browsers and shared proxy caches;
  - `private`: indicates that the response is intended for a single user and should not be stored by shared caches;
  - `no-cache`: requires caches to revalidate the resource with the origin server before reusing it;
  - `no-store`: prevents caches from storing the resource entirely.

  For example:

  ```
  Cache-Control: public, max-age=86400
  ```

  allows the resource to remain cached for one day by both browsers and intermediary proxy servers.

These headers define the resource **Time-To-Live (TTL)**, that is, the amount of time during which the cached resource is considered fresh. The most important directives provided by the `mod_expires` module are:

- `ExpiresActive On | Off`: Enables or disables expiration processing;
- `ExpiresDefault <code>seconds`: Defines the default expiration policy. The `seconds` parameter is the TTL of the resource expressed in seconds. The `<code>` parameter is a letter specifying the starting point of the TTL. The letter can be:
  - `M`: To set the starting point of the TTL to the last modification time of the resource. We can use this option if we want the resource to expire at the same time for all users. We can use this option, for instance, for weekly notices located at the same URL;

  - A: To set the starting point of the TTL to the time the client accesses the resource. We can use this option if we want the resource to expire after a fixed amount of time to the user's perspective even if it implies that the resource has a different expiration date for each user. This can be good for image files that don't change very often, particularly for a set of related documents that all refer to the same images (i.e., the images will be accessed repeatedly within a relatively short timespan).

  It is also possible to use strings in substitution of the <code>seconds parameters for human readability, for instance:

  - "modification plus 5 hours 3 minutes"
  - "access plus 1 month 15 days 2 hours"

- ExpiresByType MIME-type <code>seconds: Defines expiration policies according to MIME types. The <code> and seconds parameters are the same as the previous directive. We use this directive if we have different needs for the different file types. For example, static images may remain valid for several days, while HTML pages may require shorter expiration times.

When a cached resource becomes stale, the client may contact the server to validate its local copy instead of downloading the entire resource again.

Here is an example of configuration file using these directives:

```
# Enable expirations
ExpiresActive On

# Default fallback value for all unspecified MIME-types
ExpiresDefault M2592000 # One month after modification

# Expire GIF images after a month in the client's cache
ExpiresByType image/gif A2592000

# HTML documents are good for a week from the
# time they were changed
ExpiresByType text/html M604800

# It is possible to use wildcards characters to select
# all extensions for a particular file type
ExpiresByType video/* "access plus 1 month"
```

5.3.3 Entity Tags (ETags)

Apache also supports cache validation through **Entity Tag (ETag)** response headers. An ETag is a unique identifier associated with a specific version of a resource. Apache can generate ETags for static files using the FileETag component ... directive, which specifies which file attributes should contribute to the ETag generation process. Apache may generate ETags using one or a combination of the following components:

- Size: File size;
- MTime: The date and time the file was last modified will be included;

- `INode`: Inode number (index node number, a unique integer identifier for files and directories in Unix-like file systems);
- `All`: All available fields will be used. This is equivalent to:

```
FileETag INode MTime Size
```

- `None`: If a document is file-based, no ETag field will be included in the response.

When a client already possesses a cached version of a resource, it may send the previously received ETag back to the server using the HTTP header:

```
If-None-Match: "<etag_value>"
```

Apache then compares the received ETag with the current version of the resource:

- if the resource has not changed, the server replies with `304 Not Modified` and no response body is transmitted;
- if the resource has changed, Apache sends the updated content together with a new ETag.

This mechanism significantly reduces:

- bandwidth usage;
- response times;
- unnecessary data transfers.

5.3.4 Example of caching configuration file

The following snippet shows an example of caching configuration file.

```
LoadModule cache_module modules/mod_cache.so
<IfModule mod_cache.c>
        LoadModule cache_disk_module modules/mod_cache_disk.so
        <IfModule mod_cache_disk.c>
                CacheRoot c:/cacheroot
                CacheEnable disk /
                CacheMinFileSize 64
                CacheMaxFileSize 640000
        </IfModule>

        CacheDisable http://www.example.com/update/
</IfModule>
```

In this file we first load `cache_module` and then `cache_disk_module` and we use the `<IfModule>` sectioning directive to make sure that the modules were loaded before we proceed.

We use some previously seen directives inside the nested sectioning directive because they require both modules to be used and we use the directive `CacheDisable` to avoid storing in the cache the files in the folder `http://www.example.com/update/`. The files in this folder may be either too big, updated frequently (dynamic content), sensitive information or infrequently

requested content. Considering that the last directive requires only the `cache_module` module but not the other, we can use this directive inside `<IfModule mod_cache.c>` but outside of `<IfModule mod_cache_disk.c>`.

5.4 Apache as a Proxy Server

Apache HTTP Server can be configured to operate as a proxy server, acting as an intermediary between clients and other servers. Depending on the deployment scenario, Apache may operate either as:

- a **forward proxy**, serving clients that wish to access external resources;
- a **reverse proxy** (or gateway), serving backend servers hidden behind Apache.

These features are mainly provided by the `mod_proxy` module and its related extensions.

5.4.1 Forward Proxy

A forward proxy is a server used by clients on an internal network to access resources located on external networks such as the Internet. In this configuration:

- the client explicitly knows that it is communicating with a proxy;
- the proxy receives the client's requests;
- the proxy forwards the requests to the destination server;
- the proxy receives the response and sends it back to the client.

The overall communication flow can therefore be summarized as:

$$\text{Client} \rightarrow \text{Forward Proxy} \rightarrow \text{External Server}$$

$$\text{Client} \leftarrow \text{Forward Proxy} \leftarrow \text{External Server}$$

Forward proxies are commonly used for:

- centralized Internet access control;
- content filtering;
- traffic monitoring;
- anonymization of client requests;
- caching frequently requested resources.

Forward proxying is enabled using the directive:

```
ProxyRequests On
```

Apache also provides the directive:

```
ProxyVia On
```

which controls the generation of the HTTP `Via` header. This header identifies intermediate proxy servers involved in handling the request and response. Proxy access control rules are typically configured using the `<Proxy>` container. For example:

```
ProxyRequests On
ProxyVia On

<Proxy "*">
        Require host internal.example.com
</Proxy>
```

In this example:

- the wildcard `"*"` indicates that the configuration applies to all proxied resources;
- only hosts belonging to the domain `internal.example.com` are authorized to use the proxy.

Without appropriate access restrictions, a forward proxy may become an **open proxy**, which can be abused for malicious activities.

5.4.2 Reverse Proxy (Gateway)

A reverse proxy operates in the opposite direction. Instead of representing clients, it represents one or more backend servers. In this configuration:

- clients communicate only with the reverse proxy;
- the reverse proxy forwards requests to backend servers;
- backend servers generate the responses;
- the reverse proxy returns the responses to the clients.

The communication flow becomes:

$$\text{Client} \rightarrow \text{Reverse Proxy} \rightarrow \text{Backend Servers}$$

$$\text{Client} \leftarrow \text{Reverse Proxy} \leftarrow \text{Backend Servers}$$

To the client, the reverse proxy appears to be the actual origin server. The client is generally unaware of the existence of the backend infrastructure. Reverse proxies are commonly used for:

- load balancing;
- hiding internal infrastructure;
- centralized SSL/TLS termination;
- request filtering and security;
- caching;
- improving scalability and fault tolerance.

The most important directives for configuring a reverse proxy are:

```
ProxyPass "/foo" "http://example.com/bar"
ProxyPassReverse "/foo" "http://example.com/bar"
```

The directive `ProxyPass` forwards requests matching a specific URL path to a backend server. For example:

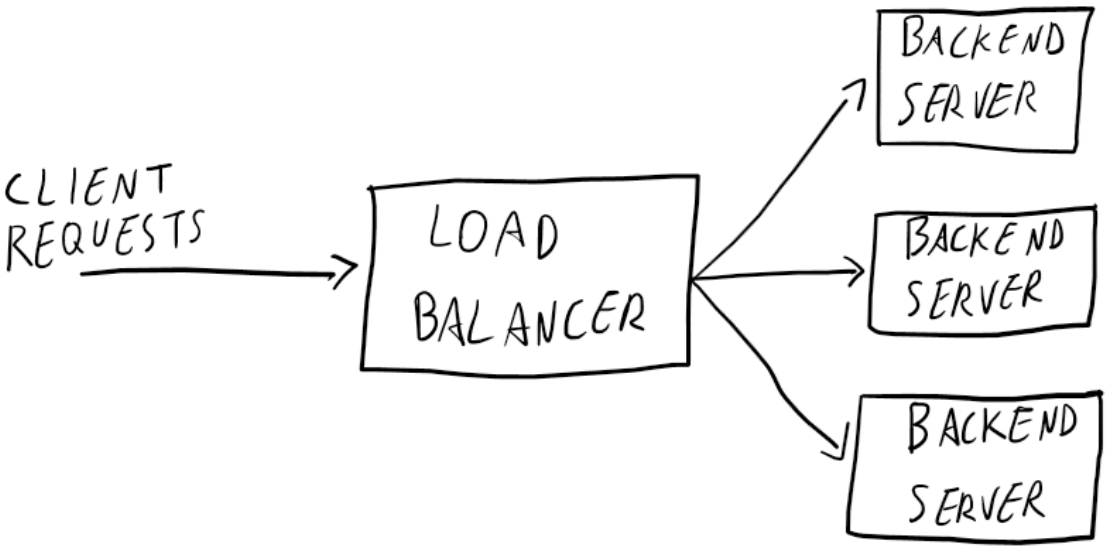

Figure 5.3: Load balancer distributing the client requests across three backend servers

```
ProxyPass "/api" "http://backend.internal/api"
```

causes requests directed to:

```
/api
```

to be internally forwarded to the backend server.

The directive `ProxyPassReverse` is used to rewrite HTTP redirection headers generated by the backend server. In particular, it adjusts:

- `Location`;
- `Content-Location`;
- `URI`

headers contained in redirect responses so that clients continue interacting transparently with the reverse proxy rather than directly with backend servers.

5.4.2.1 Load Balancing

One of the most important applications of reverse proxies is **load balancing**. In this architecture, Apache acts as a gateway and distributes incoming client requests across multiple backend servers (see figure 5.3). Each backend server processes a subset of the incoming traffic. This architecture provides several advantages:

- improved scalability;
- better fault tolerance;
- higher availability;

- reduced workload on individual servers.

Since clients communicate only with the reverse proxy, changes to the backend infrastructure remain transparent to users. Backend servers may therefore:

- be added or removed dynamically;
- use different hardware configurations;
- run different application instances;
- be geographically distributed.

A simplified Apache load balancing configuration is shown below:

```
<Proxy "balancer://mycluster">
        BalancerMember "http://server1.internal"
        BalancerMember "http://server2.internal"
</Proxy>

ProxyPass "/" "balancer://mycluster/"
ProxyPassReverse "/" "balancer://mycluster/"
```

In this example:

- the `<Proxy>` container defines a logical cluster named `mycluster`;
- the `BalancerMember` directives specify the backend servers participating in the cluster;
- incoming requests are distributed among the backend servers by Apache.

This mechanism allows the system to handle larger workloads while maintaining a single public entry point for clients.

5.5 Securing Connections with SSL/TLS

In the modern web, encrypting all traffic with SSL/TLS (Secure Sockets Layer/Transport Layer Security) is a fundamental best practice. Widespread risks like eavesdropping and user tracking make unencrypted communication unsafe, even for content that isn't explicitly sensitive. In fact, we may think that we are not sharing personal information when we browse the internet without inserting data but we actually are! Websites aim to gather as much fingerprint information as possible from the clients so we need a way to encrypt **all** our communications. Apache's `mod_ssl` module provides an interface to the OpenSSL library, enabling strong encryption.

5.5.1 Core Configuration

Enabling a basic TLS configuration for a virtual host requires a few essential directives:

- `Listen 443`: Instructs the server to listen on port 443, the standard for HTTPS traffic;
- `SSLEngine on`: Enables the TLS engine for the specified virtual host;
- `SSLCertificateFile /path/to/server.crt`: Specifies the path to the server's public certificate file, which is presented to clients;
- `SSLCertificateKeyFile /path/to/server.key`: Specifies the path to the server's private key, which must be kept secure;

The following example contains all the directives we just exposed:

```
LoadModule ssl_module modules/mod_ssl.so
Listen 443
<VirtualHost *:443>
        ServerName www.example.com
        SSLEngine on
        SSLCertificateFile "/path/to/www.example.com.cert"
        SSLCertificateKeyFile "/path/to/www.example.com.key"
</VirtualHost>
```

5.5.2 Generating a self-signed TLS certificate using the Linux `openssl` command

TLS certificates are generally generated by an external Certification Authority (CA) but, for testing and development purposes, you can act as your own Certificate Authority (CA) and generate a self-signed certificate. This is not suitable for production but allows you to test TLS configurations locally.The following `openssl` commands generate the necessary files:

1. Create the CA's private key:

   ```
   openssl genrsa -des3 -out myca.key 4096
   ```

2. Create the CA's public certificate:

   ```
   openssl req -new -x509 -days 365 -key myca.key -out myca.crt
   ```

3. Create the server's private key:

   ```
   openssl genrsa -des3 -out server.key 4096
   ```

4. Create a Certificate Signing Request (CSR) for the server:

   ```
   openssl req -new -key server.key -out server.csr
   ```

5. Use the CA to sign the server's CSR and create the final public server certificate:

   ```
   openssl req -x509 -days 365 -in server.csr -CA myca.crt \
           -CAkey myca.key -set_serial 01 -out server.crt
   ```

5.5.3 Self-Signed Certificates vs External Certification Authorities

When configuring HTTPS on a web server, administrators must obtain a digital certificate associated with the server identity. This certificate may either be **self-signed** or issued by an external **Certification Authority (CA)**. In the case of a **self-signed certificate**, the administrator independently performs the entire certificate generation process. This includes:

- creating the Certification Authority key;
- creating the Certification Authority certificate;
- creating the Certificate Signing Request;
- generating the server private key;
- signing the server certificate.

In this model, the same entity acts both as the server owner and the trusted authority certifying the server identity. Although self-signed certificates are simple to generate and useful for:

- local development;
- internal networks;
- testing environments;

they are generally not trusted by browsers because no publicly recognized Certification Authority validates the server identity.

Conversely, when using an **external Certification Authority**, part of the certificate generation process is delegated to a trusted third-party organization. In this scenario, the administrator usually generates a:

- server private key;
- Certificate Signing Request (CSR).

The CSR contains:

- the server public key;
- identification information about the organization or domain owner.

The CSR is then sent to the external CA, which verifies the identity of the requester before issuing the signed certificate.

The verification process may involve:

- domain ownership validation;
- organizational verification;
- legal documentation;
- digital or paper-based contracts.

Certification Authorities require the requester to sign documents or provide proof of identity in order to guarantee that certificates are issued only to trusted entities. Some authorities perform the entire procedure digitally, while others may still require physical documentation depending on the certificate type and validation level.

The exact certificate issuing workflow may vary depending on the policies of the Certification Authority. For example:

- the CA may directly generate and sign the server certificate before sending it to the customer;
- the CA may instead provide only a signed CSR, leaving part of the key generation process to the customer.

The main advantage of using an external CA is that modern browsers and operating systems already trust well-known Certification Authorities through pre-installed root certificates. Consequently, users accessing the website do not receive security warnings, unlike what typically happens with self-signed certificates.

While Apache's modularity, flexibility, and rich feature set have made it a cornerstone of web infrastructure, its traditional process-driven architecture presented challenges under the extreme loads of the modern web. The industry's search for a solution to the "C10k problem"—the challenge of efficiently handling ten thousand concurrent connections—led to the development of fundamentally different server architectures. Nginx emerged as a direct architectural response, designed from the ground up for high concurrency and resource efficiency.